# Documento de diseño — *Nidada*

**Juego de mascotas virtuales que sirve de banco de pruebas de `godot-chain`.**

Versión 0.1 — propuesta, 5 de octubre de 2026. Pendiente de revisión.

---

## 0. Estatus de este documento — léelo antes que el resto

Este documento describe **el juego completo al que aspira la demo**, no lo que se construye en las 40 horas del PFM. Tiene el mismo estatus que el `Documento de implementación.md`: **referencia, no guía de acción**.

| Qué | Dónde se decide |
|---|---|
| Lo que se implementa en el PFM | Sección 7 de la propuesta reducida, y la **sección 10** de este documento, que la traduce a *Nidada* sin añadir nada |
| Lo que se implementa después del PFM | El resto de este documento, si algún día se implementa |

Si una mecánica de aquí exige algo que la propuesta o el contrato HTTP no ofrecen, **no se amplía el contrato**: se rediseña la mecánica o se apunta en la sección 13. El juego se adapta a la librería, no al revés. Precisamente eso es lo que el juego tiene que demostrar: que una librería con cuatro operaciones y activos fungibles basta para un juego con mascotas únicas.

---

## 1. Concepto

> Cuidas a un puñado de criaturas pequeñas. Crecen en tiempo real. Cuando son adultas, puedes presentarlas a las mascotas de otros jugadores, y de ese encuentro nace un huevo.

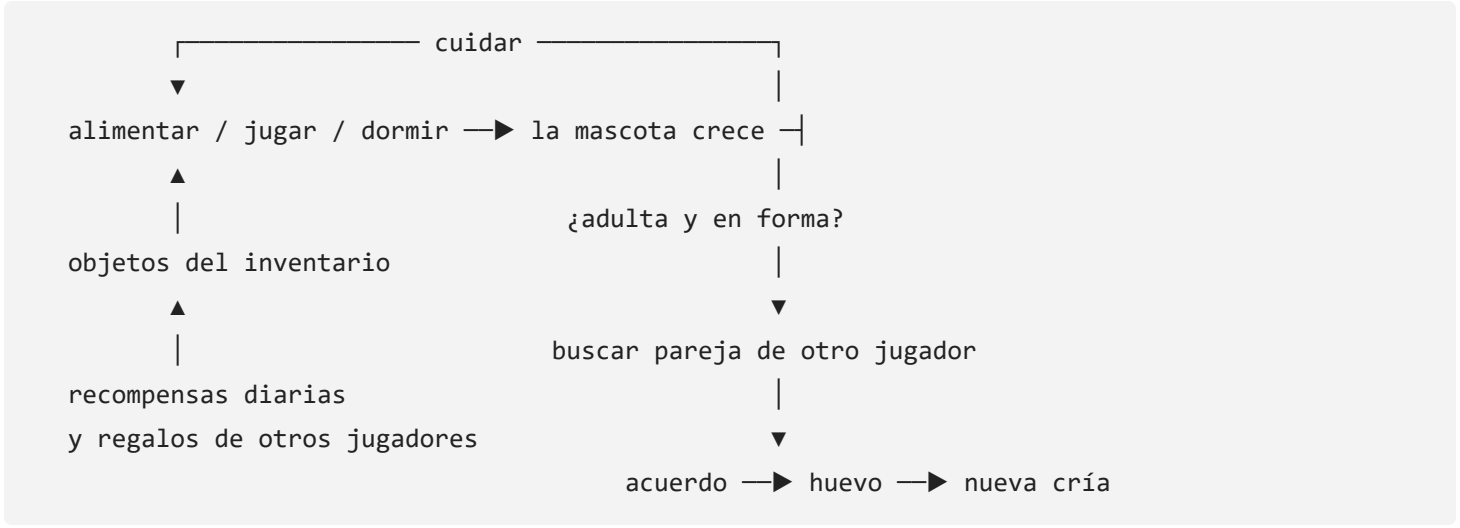
**Género:** mascota virtual (*virtual pet*) con componente social asíncrono. **Referentes:** Tamagotchi (cuidado en tiempo real), Neopets (colección y comunidad), Pokémon (guardería, cría entre especies). **Sesión típica:** 3–5 minutos, varias veces al día. **Plataforma objetivo:** escritorio y Android. **No Steam** (ver el aviso de la sección 16 de la propuesta: Steam veta los juegos que permiten intercambiar activos de cadena).

### 1.1 Pilares

1. **Cada mascota es tuya y es única.** Tiene nombre, color, fecha de nacimiento y padres.
2. **Lo valioso es raro y está certificado.** Las especies y los objetos de colección existen fuera del juego; el estudio no puede inflarlos en silencio.
3. **La única forma de conseguir ciertas especies es con otra persona.** La cría entre jugadores es el motor social, no un añadido.
4. **El jugador no ve la cadena.** Ni *wallet*, ni comisiones, ni palabras técnicas. Lo más que ve es un sello de «registrado» cuando algo es definitivo.

---

## 2. Bucle de juego




- **Bucle corto (minutos):** atender necesidades, usar objetos, ver la reacción.
- **Bucle medio (días):** llevar crías a adultas sin descuidarlas.
- **Bucle largo (semanas):** completar el catálogo de especies, que exige cruces con otros jugadores porque nadie empieza con todas.

---

## 3. Mascotas

### 3.1 Especie e individuo — la distinción que lo sostiene todo

Una mascota tiene **dos capas**, y esta separación no es estética: es la única forma de encajar mascotas únicas en una librería de activos **fungibles** (decisión 4.1 y limitación 12.1 de la propuesta).

| Capa | Ejemplo | Dónde vive | Qué garantiza |
|---|---|---|---|
| **Especie** | «Brasa», rareza común, elemento fuego | Un `Item` de `godot-chain` (un tipo) | Que la especie existe, cuántos ejemplares hay y **quién tiene cuántos** |
| **Individuo** | «Chispi», naranja con manchas, nacida el 3 de octubre, hija de X e Y, hambre 40 % | Base de datos del servidor de juego | Nada fuera del juego. Es estado del juego, como lo sería en cualquier otro |

**Invariante de reconciliación:** para todo jugador *J* y especie *E*,

```
nº de individuos de E cuyo dueño es J   ==   amount de E en el inventario de J
```

El servidor de juego lo comprueba al iniciar sesión del jugador (una lectura de inventario) y al recibir cada `settled`. Si no cuadra, manda la cadena: el servidor marca la discrepancia y no permite operar con esa especie

hasta que un humano la revise. Nunca crea ni borra individuos para «arreglar» la cuenta.

### 3.2 Catálogo de especies

Diez especies, creadas de antemano por un script de carga. Ninguna se crea durante la partida.

| Especie | Elemento | Rareza | Cómo se obtiene |
|---|---|---|---|
| Brasa | fuego | común | Inicial |
| Gota | agua | común | Inicial |
| Musgo | planta | común | Inicial |
| Pluma | aire | común | Inicial |
| Vaho | fuego + agua | rara | Cría Brasa × Gota |
| Ceniza | fuego + planta | rara | Cría Brasa × Musgo |
| Chispa | fuego + aire | rara | Cría Brasa × Pluma |
| Ciénaga | agua + planta | rara | Cría Gota × Musgo |
| Bruma | agua + aire | rara | Cría Gota × Pluma |
| Polen | planta + aire | rara | Cría Musgo × Pluma |

Cada jugador nuevo recibe **dos** mascotas iniciales de **dos especies comunes distintas**, elegidas al azar entre las cuatro. Así nadie tiene las cuatro, y cualquier especie rara exige al menos una especie que el jugador no tiene: es decir, otro jugador.

**Cualidades de tipo** ( `qualities` del `Item`, máximo 8 según el contrato):

| Clave | Valores |
|---|---|
| `kind` | `pet` |
| `element` | `fire`, `water`, `plant`, `air`, o dos unidos por `+` |
| `rarity` | `common`, `rare` |
| `growth_hours` | Horas reales hasta adulta (72 en comunes, 96 en raras) |

Son las mismas para todos los ejemplares de la especie, como exige la fungibilidad. Lo que varía por ejemplar (color, patrón, nombre) **no** es una cualidad: es estado del individuo.

### 3.3 Rasgos individuales

Estado del servidor de juego, cosmético, no certificado:

| Rasgo | Valores | Herencia |
|---|---|---|
| Color | 6 tonos | 45 % el de un progenitor, 45 % el del otro, 10 % mutación aleatoria |
| Patrón | liso, manchas, rayas | Igual que el color, con 5 % de mutación |
| Nombre | Texto libre, 1..16 caracteres, filtrado | Lo elige el dueño al eclosionar |
| Linaje | IDs de los dos progenitores | Fijo al nacer |

El linaje es sólo informativo. **No hay penalización por consanguinidad**: complica el diseño y no prueba nada de la librería.

### 3.4 Etapas de vida

| Etapa | Duración (comunes) | Puede criar | Notas |
|---|---|---|---|
| Huevo | 1 h | no | Aparece en el corral al asentarse el nacimiento (ver 6.4) |
| Cría | 0–24 h | no | Necesidades bajan el doble de rápido |
| Joven | 24–72 h | no | |
| **Adulta** | ≥ 72 h | **sí**, si su bienestar ≥ 60 % | Etapa final. No hay vejez ni muerte |

**Adulta** significa: edad ≥ `growth_hours` de su especie **y** bienestar medio de las últimas 24 h ≥ 60 %. Una mascota descuidada llega a la edad pero no puede criar hasta recuperarse. Así la cría premia el cuidado y no sólo el paso del tiempo.

**Por qué no hay muerte.** La desaparición de un individuo obligaría a hacer desaparecer una unidad de la especie, y el contrato no tiene operación de destrucción (ver 13.2). Una mascota descuidada se pone triste y deja de crecer; no muere. Además es mejor diseño para el público.

**Escala de tiempo configurable.** El servidor de juego lleva un factor de aceleración ( `TIME_SCALE` ). En producción vale 1; en la demo, 120 (72 h → 36 min).

### 3.5 Necesidades

Tres barras de 0 a 100 que bajan con el tiempo real y se calculan en el servidor a partir de la marca temporal de la última interacción; no hace falta tener el juego abierto.

| Necesidad | Baja | Se repone con |
|---|---|---|
| Hambre | 4 puntos/h | Comida (objeto consumible) |
| Ánimo | 3 puntos/h | Jugar (gratis, 1 vez cada 30 min) o un juguete |
| Energía | 2 puntos/h despierta | Dormir (la mascota no está disponible 2 h) |

Bienestar = media de las tres. Por debajo de 30 % la mascota deja de crecer.

---

# 4. Inventario y objetos

## 4.1 Dos clases de objeto

| Clase | Ejemplos | ¿En `godot-chain`? | Motivo |
|---|---|---|---|
| **Consumibles** | Pienso, Fruta, Pescado | **No.** Contador en el servidor de juego | Se gastan varias veces al día. Cada consumo sería una transacción, y el contrato no tiene operación para destruir unidades (13.2). Además el valor de certificar un pienso es nulo |
| **Coleccionables** | Galleta de cortejo, Lazo dorado, Cascabel de luna | **Sí.** Cada uno es un `Item` | Escasos, duraderos, intercambiables entre jugadores. Es exactamente lo que la librería sirve |

Regla para decidir en el futuro: **si el objeto se gasta, no es un `Item`.** Si se conserva y se puede regalar, sí.

## 4.2 Coleccionables del catálogo inicial

| Objeto | Cualidades | Uso |
|---|---|---|
| Galleta de cortejo | `kind=treat`, `rarity=common` | Pago para pedir una cría (6.3). Se recibe una por día de juego |
| Lazo dorado | `kind=accessory`, `rarity=rare`, `slot=neck` | Decorativo. Recompensa del primer nacimiento |
| Cascabel de luna | `kind=accessory`, `rarity=rare`, `slot=neck` | Decorativo. Recompensa por completar las cuatro especies comunes |

Los accesorios se «equipan» a un individuo, pero equipar es **estado del servidor de juego**: la unidad sigue en el inventario del jugador, sólo cambia a qué mascota se dibuja encima. Mientras está equipado, el servidor no permite regalarlo.

### 4.3 Regalar

Un jugador puede regalar **coleccionables** a otro: es una transferencia de `godot-chain`. **Las mascotas no se regalan ni se venden.** On-chain serían transferibles (`NonTransferable` no está activado, limitación 12.4), pero el servidor de juego nunca lo pide: mover una especie sin mover el individuo rompería el invariante de 3.1, y moverlo bien exige una coordinación entre servidor y cadena que no aporta nada a la demo.

---

## 5. Interacción entre jugadores

Asíncrona. No hace falta que los dos estén conectados a la vez salvo para aceptar una cría.

| Acción | Qué hace | Toca la cadena |
|---|---|---|
| Visitar | Ver el corral de otro jugador: sus mascotas, edades, rasgos | No (lectura del servidor de juego) |
| Regalar | Enviar un coleccionable | Sí: `transfer` |
| Proponer cría | Pedir que una mascota propia se junte con una ajena | Sí, al aceptar: `transfer` + `grant` |

Los jugadores se encuentran por **código de amigo** (6 caracteres) que gestiona el servidor de juego. No hay chat, ni gremios, ni búsqueda pública: identidad y social están fuera del alcance de la librería (premisa 2 de la propuesta) y no prueban nada de ella.

---

## 6. Cría

La mecánica central, y la que más exige a la librería: dos operaciones encadenadas, dos jugadores, estado intermedio que puede fallar.

### 6.1 Requisitos

Una cría entre la mascota **M1** del jugador **A** y la mascota **M2** del jugador **B** se permite si y sólo si:

1. A ≠ B. **La cría es siempre entre jugadores distintos.** Criar con uno mismo vaciaría el pilar 3.
2. M1 y M2 son **adultas** (definición de 3.4).
3. Ninguna de las dos ha criado en las últimas **48 h** (tiempo de juego escalado).
4. Ninguna de las dos ha alcanzado su límite de **3 crías** en toda su vida.
5. A tiene al menos **1 Galleta de cortejo** que no esté reservada por otra cría en curso.
6. No hay otra cría en curso en la que participe M1 o M2.

Las reglas 3 y 4 son el freno a la inflación: sin ellas, dos amigos generarían especies raras sin límite y la rareza dejaría de significar algo.

## 6.2 Quién recibe la cría

**El que propone (A) se queda la cría. El que acepta (B) se queda la galleta.**

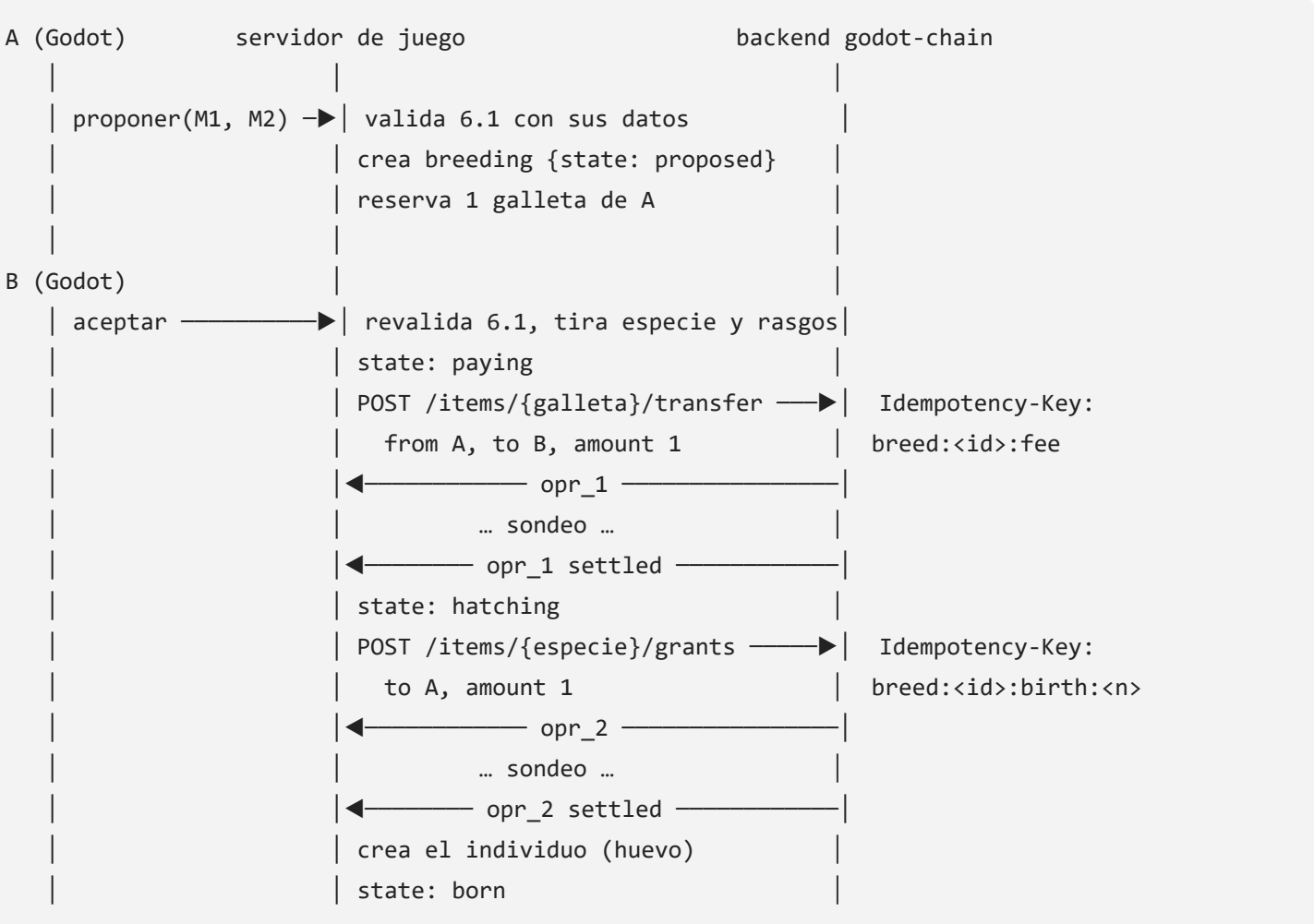
Es un intercambio con sentido para los dos: A paga para obtener una especie que no podía tener; B cobra por prestar su mascota. Y en términos de la librería, cada mitad es una operación que ya existe: la galleta es un `transfer`, la cría es un `grant`.

## 6.3 Resultado del cruce

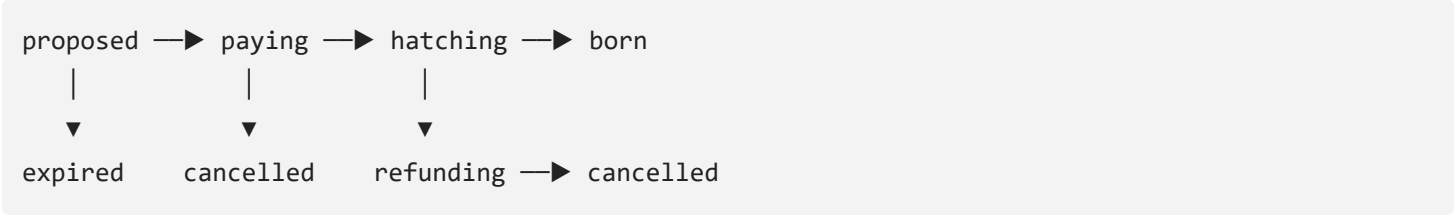
| Progenitores | Especie de la cría |
|---|---|
| Misma especie común (Brasa × Brasa) | Esa especie |
| Dos comunes distintas | La rara de la tabla 3.2 |
| Al menos una rara | La de uno de los dos progenitores, al 50 % |

Los rasgos individuales se heredan según 3.3. La tirada la hace el servidor de juego **al aceptar**, y queda guardada en la cría en curso: reintentar no vuelve a tirar.

## 6.4 Flujo



**Máquina de estados de la cría** (servidor de juego, no librería):




| Transición | Cuándo |
|---|---|
| `proposed → expired` | B no responde en 10 min, o rechaza. Se libera la galleta reservada |
| `paying → cancelled` | La transferencia de la galleta acaba en `failed` no reintentable (p. ej. `INSUFFICIENT_BALANCE`) |
| `paying → paying` | `failed` reintentable: se reintenta con clave nueva, como exige el contrato (8, «reglas transversales») |
| `hatching → hatching` | El nacimiento falla con error reintentable: se reintenta con `<n>+1` |
| `hatching → refunding` | El nacimiento falla con error no reintentable: se devuelve la galleta (`transfer` de B a A, clave `breed:<id>:refund`) |

**Por qué dos pasos y no uno.** El contrato no tiene operaciones en lote ni atómicas (sección 11 de la propuesta). Cobrar primero y dar después es el orden seguro: si el cobro falla, no ha pasado nada; si el nacimiento falla, hay una galleta que devolver. El orden inverso dejaría una cría regalada si después el cobro fallase.

**Por qué el estado se escribe en el servidor antes de llamar.** Es la misma regla que la librería aplica dentro (invariante 2): el servidor de juego guarda `paying` y la clave antes de mandar la transferencia. Si se cae entre medias, al volver repite la petición con la misma clave y el backend le devuelve la operación original en vez de cobrar dos veces.

## 6.5 Qué ve cada jugador

Aquí es donde `applied` y `settled` se convierten en experiencia de juego.

| Señal del `ChainHandle` | Pantalla de A | Pantalla de B |
|---|---|---|
| Galleta `applied` | La galleta sale volando del inventario de A (animación) | La galleta aparece con brillo apagado |
| Galleta `settled` | — | La galleta se asienta; sello «registrado» |
| Nacimiento `applied` | Aparece un huevo **temblando y translúcido** en el corral | — |
| Nacimiento `settled` | El huevo se vuelve opaco y empieza la cuenta atrás de eclosión | Notificación: «La cría de M2 y M1 ha nacido en casa de A» |
| Cualquier `failed` | La animación se deshace; mensaje sin tecnicismos («No se pudo completar, tu galleta vuelve a ti») | Ídem |

**El juego sólo persiste en `settled`** (invariante 5). El huevo translúcido de `applied` no se puede tocar, no cuenta para ningún logro y desaparece si llega `failed`. Esta pantalla es la demostración más visual de por qué la librería distingue los dos estados.

Tiempo esperable en devnet entre proponer y huevo opaco: del orden de decenas de segundos (dos operaciones hasta finalización). **Medirlo** en la demo; no prometer cifras antes.

---

## 7. Qué vive dónde

Resumen de toda la sección anterior en una tabla. Es la que hay que defender en la presentación.

| Dato | Cliente Godot | Servidor de juego | `godot-chain` / cadena |
|---|---|---|---|
| Identidad del jugador (`external_id`) | La recibe del login | **La crea** | La recibe, no la verifica |
| Catálogo de especies y coleccionables | Lo lee | Conoce los `item_id` | **Fuente de verdad** |
| Cuántas mascotas de cada especie tiene cada jugador | Lo lee | Lo reconcilia | **Fuente de verdad** |
| Cada individuo: nombre, color, edad, padres, necesidades | Lo dibuja | **Fuente de verdad** | Nada |
| Cuántos coleccionables tiene cada jugador | Lo lee | Lo consulta | **Fuente de verdad** |
| Accesorio equipado | Lo dibuja | **Fuente de verdad** | Nada |
| Consumibles | Los muestra | **Fuente de verdad** | Nada |
| Crías en curso y sus reglas | Las muestra | **Fuente de verdad** | Nada |
| Decidir si una cría se permite | Nunca | **Siempre** | Nunca |
| `write_key` | **Nunca** | Sí | La valida |

La columna del cliente no decide nada. Un cliente modificado puede mostrarse a sí mismo diez especies raras, pero no puede conseguir que el servidor pida un `grant`, que es lo único que crea una mascota de verdad (sección 3 de la propuesta).

---

## 8. Correspondencia con `godot-chain`

| Concepto del juego | Concepto de la librería | Endpoint |
|---|---|---|
| El juego *Nidada* | Un despliegue del backend (`game_id`) | — |
| «Especies» y «Coleccionables» | Dos `Collection` | `POST /v1/collections` |
| Una especie, un coleccionable | Un `Item` | `POST /v1/items` (script de carga) |
| Elemento, rareza, tipo | `qualities` | Dentro de `POST /v1/items` |
| Jugador | `Holder` (`external_id` → `hld_`) | — |
| Mascotas iniciales, galleta diaria, nacimiento, recompensas | Emisión | `POST /v1/items/{id}/grants` |
| Regalo, pago de cría, devolución | Transferencia | `POST /v1/items/{id}/transfer` |
| Corral e inventario certificados | Inventario | `GET /v1/holders/{id}/items` |
| Huevo translúcido / huevo opaco | `applied` / `settled` | `GET /v1/operations/{id}` |

**Nada de esta tabla exige ampliar el contrato.** Es la comprobación de que el diseño cabe.

**Dependencia crítica:** casi todo el juego depende de `POST /grants`, que es el **punto 2 de la línea de corte** (8.1 de la propuesta). Si se recorta, *Nidada* se queda sin nacimientos, sin galleta diaria y sin mascotas iniciales para jugadores que no estuvieran en el script de carga. No afecta al PFM (el hito no lo necesita), pero sí a cualquier versión jugable del juego.

**Coste en cadena de la carga inicial:** 10 especies + 3 coleccionables = 13 tipos. Cada uno paga la renta de un tipo con metadatos, y cada jugador nuevo, la de sus cuentas de inventario. Las cifras son el punto **13.5 pendiente de verificar** de la propuesta; no se estiman aquí.

---

## 9. Economía

| Fuente (entra) | Sumidero (sale) |
|---|---|
| 2 mascotas iniciales por jugador | — (las mascotas no salen nunca) |
| 1 Galleta de cortejo por día de juego | Pasa de un jugador a otro al criar: **no sale**, circula |
| Crías: máx. 3 por mascota, 1 cada 48 h | — |
| Recompensas de logros (accesorios) | — |

**No hay sumideros.** Es una consecuencia directa de que el contrato no tenga operación de destrucción (13.2). La inflación se contiene sólo por el lado de las fuentes: límites de cría, galleta diaria única, y el hecho de que las especies raras exijan cooperación.

Consecuencia aceptada: a largo plazo la Galleta de cortejo se acumula y pierde valor. En un juego de una sesión de demo es irrelevante; en uno vivo, sería lo primero que habría que resolver.

**No hay tienda ni moneda.** Ni con dinero real ni con moneda de juego. Una moneda sería otro `Item` y una tienda, otro sumidero que no existe; además es exactamente el tipo de cosa que pone el juego en problemas con las tiendas de aplicaciones.

---

## 10. Lo que se construye en el PFM — corte vertical

**Ésta es la única sección que se implementa.** Es el hito de la sección 7 de la propuesta vestido con el vocabulario de *Nidada*, paso por paso, sin añadir operaciones:

| Paso del hito | En *Nidada* |
|---|---|
| Crear una colección | El servidor de juego crea la colección «Coleccionables» |
| Crear un objeto con cualidades de tipo | Crea la **Galleta de cortejo** ( `kind=treat` , `rarity=common` ) con `initial_grant` de 3 unidades al jugador A |
| Entregarlo al jugador A | Lo hace el propio `initial_grant` . No hace falta `POST /grants` |
| Transferir una unidad de A a B | A **regala** una galleta a B: el primer paso de una cría (6.4, «paying») |
| Ver el inventario de ambos actualizado | Dos paneles, A y B, con la galleta y su cantidad; el animado en `applied` y el sello en `settled` |

**La demo es una escena con dos paneles**, uno por jugador, y un selector de jugador en lugar de login. Lleva el sprite de la galleta y, como mucho, una mascota estática de decorado por panel para dar contexto. **No hay** individuos, necesidades, crecimiento ni cría jugable: la cría se cuenta en el vídeo como «lo que este pago permite», con la sección 6.4 como diagrama.

Si se aplica el punto 4 de la línea de corte, la escena pierde los sprites y queda con botones y `Label` , sin cambiar el guion.

**Lo que este corte demuestra del juego completo:** que el paso más delicado de la cría —mover un activo entre dos jugadores con la autoridad del juego como único firmante, y reflejarlo en dos pantallas con `applied` y `settled` — funciona de extremo a extremo. Todo lo demás de este documento es estado de servidor de juego convencional, que no necesita probarse en un PFM sobre cadena.

---

## 11. Fuera de alcance del juego

Ni en el PFM ni en este diseño:

- Muerte, vejez o abandono de mascotas.
- Regalar, vender o subastar mascotas. Mercado de cualquier tipo.
- Cría con uno mismo.
- Moneda de juego, tienda, compras con dinero real.
- Chat, gremios, búsqueda pública de jugadores, clasificaciones.
- Combates o minijuegos competitivos.
- Rasgos individuales certificados fuera del juego (exigirían activos no fungibles: sección 11 de la propuesta).
- Sacar una mascota a una cartera propia ( `claim` ).

---

## 12. Riesgos de diseño

| Riesgo | Efecto | Mitigación |
|---|---|---|
| El jugador nota la latencia del nacimiento | La cría parece lenta | El huevo translúcido de `applied` da respuesta inmediata; la eclosión de 1 h oculta el resto |
| El invariante de reconciliación (3.1) se rompe | Un jugador con más mascotas que unidades, o al revés | Comprobación al iniciar sesión y en cada `settled` ; manda la cadena; bloqueo de la especie hasta revisión humana |
| B acepta y se desconecta | Nada: tras aceptar, todo ocurre en el servidor | — |
| Dos amigos cooperan para generar raras sin fin | Las raras dejan de serlo | Límites de 6.1 (48 h, 3 crías por mascota) |
| Se pide una tienda de mascotas raras | Cambia la naturaleza del proyecto y su relación con las tiendas | Fuera de alcance (sección 11), sin excepción |

---

## 13. Tensiones con la propuesta, explícitas

Puntos donde el juego roza un límite de la librería. Se dejan escritos, no resueltos con código.

1. **Mascotas únicas sobre activos fungibles.** Resuelto con las dos capas de 3.1. El precio: la cadena certifica *cuántas Brasa tiene A*, no *que Chispi existe*. Es la limitación 12.1 de la propuesta llevada a un caso real, y hay que decirlo así en la memoria.
2. **No hay operación de destrucción.** Por eso no hay consumibles en cadena (4.1), ni muerte (3.4), ni sumideros (9). Si el juego los necesitara, la vía sin ampliar el contrato sería transferir a un tenedor

«sumidero» propiedad del juego, que no destruye nada y engorda su inventario para siempre. **No se adopta**; queda anotado como pregunta.

3. **Sin operaciones atómicas.** La cría son dos operaciones con compensación (6.4). Funciona, pero exige que el servidor de juego tenga su propia máquina de estados: el «servidor de ~150 líneas» de la propuesta no basta para el juego completo, sólo para el corte vertical.
4. **El estudio puede mover cualquier mascota.** El delegado permanente (limitación 12.3) permite al estudio quitar una especie a un jugador. *Nidada* no lo usa salvo para lo que el jugador ha pedido, pero no puede demostrarlo, y el jugador tiene que confiar en ello.
5. **Dependencia de `POST /grants`.** Ver sección 8.

---

## 14. Preguntas abiertas

Para decidir antes de que nada de esto pase de referencia a implementación:

1. ¿Se acepta que el corte vertical de la sección 10 sea toda la presencia de *Nidada* en el PFM, y que la cría se presente con diagrama y no jugable?
2. ¿Interesa al tribunal ver la máquina de estados de la cría (6.4) implementada en el servidor de juego aunque sea sin arte? Costaría horas que la propuesta no tiene.
3. Si en el futuro se quieren sumideros, ¿se amplía el contrato con una operación de destrucción o se usa el tenedor sumidero de 13.2?
4. ¿Escala de tiempo de la demo (`TIME_SCALE = 120`) o eclosión instantánea para el vídeo?